



Project Assignment

Project:

Design and develop a prototype of a Web Application with **RESTful** architectural Style.

Technologies:

- **Java/Jakarta EE Specifications** for the Backend (BE) module
- **Angular with TypeScript** for the Frontend (FE) module

Assignment overview

In this assignment, you are required to perform a **full development cycle** starting from a brief **Technical Analysis** of a specific application context. The analysis stage will be useful for the subsequent **development stage of a Web Application prototype**, and it will focus on functional and non-functional requirements, a conceptual domain model, and a set of use cases that describe the interactions between users and the system.

Objectives

- Write a **technical report** which must include all the documentation produced during this Project for summarizing all your work
- Formally and clearly describe the **application context** starting from the initial description of the assignment (*see the Appendix for details*)
- Write a **Software Requirements Specification** in order to document Functional and Non-Functional requirements
- Model the **Conceptual Domain** using a *UML Class Diagram*
- Define at least **three** primary **Use Cases** and their corresponding **User Stories** (which will be concretely developed in the prototype)
- Optionally, draw the Web Application **architecture** through a *UML Deployment Diagram*
- Ensure the system design adheres to **RESTful principles**
- Develop a **prototype** (based on your design) for the Backend module and optionally for the Frontend module with prescribed technologies and frameworks



Expected Deliverables

The **technical report** must include the following **sections**:

1. Application Context and Problem Statement

- Describe the operative domain/context of the Web Application
- Identify the target Users and their key needs
- Clearly define the problem the application aims to solve

2. Requirements Analysis

- Write the *Software Requirements Specification*
- In particular:
 - Functional Requirements, a numbered list of core features the system must implement, written in the following formal syntax
 - *FR1: The system SHALL allow Users to perform login*
 - Non-Functional Requirements, a numbered list of technological constraints (e.g., architectural style, adopted programming language, framework) and quality attributes (e.g., security, performance, usability, scalability), written in the following formal syntax
 - *NFR1: The system SHALL be designed as a RESTful architecture*

3. Conceptual Domain Model

- Draw and include a *UML Class Diagram* representing the main entities in your system and their relationships
- Accompany the diagram with short textual descriptions of the key entities and their roles (for better understanding the proposed domain)

4. Use Case Analysis

- Identify and describe at least three main *Use Cases* that represent core interactions with the system. For each *Use Case*, provide:
 - Use Case Name
 - Authorized Actors
 - Resources Involved
 - Associated REST Endpoints (e.g., *GET /api/events*, *POST /api/users*)
- Optionally, include *User Stories* written in the format:
 - As a *[role]*, I want to *[action]*, so that *[benefit]*

5. Architectural Considerations

- Briefly describe the layered architecture which separates concerns between the FE (Angular), the BE (Java/Jakarta EE), and the DBMS
 - optionally, draw a *UML Deployment Diagram*



- Highlight how RESTful principles (statelessness, resource representation, HTTP methods, etc.) will be applied in the architecture design

6. Implementation

- Briefly describe the adopted technologies and the main packages and software components of BE and FE modules

Implementation Tips

- Implement a prototype of the BE module (Java) and remember to:
 - define the Java classes for your domain (i.e., `model` package) and perform the ORM process to a Relational DataBase Management System (e.g., MySQL, PostgreSQL) instance through Java Persistence API (JPA)
 - define the Java classes for your REST endpoints (i.e., `rest` package) for implementing application main Use Cases for the application business logic
 - define the Java classes for Data Transfer Objects (i.e., `dtos` package) exchanged between Backend and Frontend
 - define the Java classes for Mappers (i.e., `mappers` package)
- Optionally, implement a prototype of the FE module (Angular) and remember to:
 - define some TypeScript classes and related HTML pages (with CSS) of main “components” which control the main UI views
 - realise some custom TypeScript widget components (i.e., special components that can be reused inside other components, like a card or a button)
 - define some TypeScript classes for “services” which consume the exposed REST services by the BE module, adopting `HttpClient` provided by Angular
- **NOTE WELL:** do not implement the underlying logic for authentication process, both for BE and FE modules
 - it is sufficient to emulate this functionality (i.e., create a mock instance of the logged user or provide open REST services)

Final Notes

- The assignment must be done as a Team
- Diagrams can be created using tools like *StarUML*, *Lucidchart* or *Draw.io*
- Use formal writing and organize your document with headings and subsections
- Be prepared to discuss your work with Prof. Enrico Vicario for the final exam with a slide-based presentation (create it with tools like *Google Presentation*, *MS PowerPoint* or *OpenOffice*)



Appendix

Application context: "Municipality management system"

Please note that this application context serves as a basis of inspiration for the technical analysis, so it is NOT mandatory to develop and build the entire Web Application with the whole context [choose your SUBSET of Use Cases]

The **Municipality of Florence** manages 5 *administrative districts*, each with its own characteristics and neighborhoods:

- **District 1 – Historic Center**, which includes the city's core with main museums, institutions, and zones like San Jacopino, Piazza Puccini, Le Cascine;
- **District 2 – Campo di Marte**, which includes residential zones like Le Cure, Coverciano, Piazza Alberti, and sports areas near the Artemio Franchi Stadium;
- **District 3 – Gavinana-Galluzzo**, located southeast, with zones like Gavinana, Bandino, Sorgane, Nave a Rovezzano, and Galluzzo;
- **District 4 – Isolotto-Legnaia**, in the southwest, with zones like Isolotto, Legnaia, Soffiano, San Lorenzo a Greve, and Cintoia;
- **District 5 – Rifredi**, in the northwest, with zones like Rifredi, Novoli, Brozzi, Peretola, Quaracchi, also with the Airport and the Careggi Hospital.

The Municipality needs to carry out a digital census of the population of resident-owned motor vehicles. Therefore, a **new management system** must be designed and developed for facilitating:

- entry and updating of back-office data (i.e. data available for speeding up the next CRUD operation for end-users). These data can be: type of vehicle (e.g. car, motorbike), type of vehicle engine (e.g. internal combustion engine, electric, hybrid), city districts (i.e. D1, D2, D3, D4, D5), etc.;
- citizen registration to the system and collection of vehicles data (e.g. ownership, type of vehicle, type of engine, license plate number);
- consultation of active Resident Parking Passes records (consider that a pass allows each resident to register up to four license plates across different zones for parking owned vehicles outside his single residence zone);
- generation of summary reports (e.g. daily, weekly, monthly) with the total number of registered parking violations for each district (grouped by type of vehicle and/or type of engine).

The system may target resident **citizens**, **administrative back-office staff** of the Municipality, and even **law enforcement officers**, providing different access levels and functionalities based on designed user roles.

This digital system should help the Municipality to monitor motor vehicles population and citizen behaviour over time *[It is reasonable to assume that some information may originate from external applications or databases that share specific data with this new system]*.